

PRODUCT DOCUMENTATION

Linear Programming Recipe & Cost Optimizer

User guide & technical documentation — least-cost food formulation with a genuine two-phase simplex solver: ingredient bounds, nutrient targets, fat-to-solids ratio and protein-share rules, cost breakdowns and compliance checks.

Creator	**Dr. Asif Ahmad**
Product	Linear Programming Recipe & Cost Optimizer — version 1.0
Category	Food Science
Published by	Asif SaaS Studio
Date	21 October 2026
Format	Single-file web application (index.html, 49.7 KB)

Private by design: everything runs on the user's device — no uploads, no accounts, no tracking. Honest by design: the solver's answer is exact for the data entered, and the app says plainly what linear programming cannot model — flavour, texture and processing behaviour.

Contents

1. Overview — the formulation cost-and-composition problem
2. How linear programming works here
3. The solver — two-phase simplex, bounds by substitution, tolerances
4. Entering ingredients — the table and CSV import
5. Setting targets — including the ratio and protein-share rules
6. Reading the results — recipe, cost breakdown, compliance, binding notes
7. When there is no answer — the infeasibility guide
8. The presets — with the beef burger worked example
9. Honest scope and limitations
10. How correctness was verified — testing summary
11. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)
12. Product information and credits
13. Appendix — the tested core, verbatim

1. Overview — the formulation cost-and-composition problem

Almost every manufactured food is a compromise written in numbers. A burger blend must carry enough protein to justify its label, stay under a fat ceiling, hold its moisture, and draw a required share of its protein from meat rather than cheaper extenders — and it must do all of that at the lowest possible cost per kilogram, because a few cents per kilo decide whether a product survives. The same tension appears in flour blending (hit a protein specification from wheats and fortifiers of different prices), in drink mixes, in animal feed, in institutional catering: several ingredients, each with a price and a composition, must be combined in proportions that satisfy a set of compositional rules at minimum cost.

Doing this by hand means trial and error: adjust one ingredient, watch two targets move, adjust another, watch the first target slip. The search space is far too large for intuition once more than three or four ingredients and constraints are involved — yet the problem has a special structure that makes it exactly solvable. Cost and proximate composition (protein, fat, moisture, carbohydrate) blend linearly: the protein in a blend is simply the weighted average of the ingredients' protein contents. Problems of the form "minimise a linear cost subject to linear constraints" are linear programmes, and they have been solved exactly since the 1940s by the simplex method.

This app brings that solver to a single screen, for food R&D and nutrition formulation teams, students learning formulation mathematics, and small producers who currently formulate by spreadsheet and guesswork. With it you can:

- Enter 2–12 ingredients with cost per kg, protein / fat / moisture / carbohydrate composition, min/max inclusion bounds, and optional group tags (A / B).
- Set targets: minimum and/or maximum for each nutrient, a fat-to-solids ratio window, and a rule that at least a chosen share of the protein must come from Group A ingredients.
- Optimise, and receive the least-cost recipe: each ingredient's percentage and kilograms in your batch, its cost contribution, the achieved composition checked target-by-target, and notes on exactly which constraints are binding — i.e. which requirements are driving the cost.
- Import ingredient lists from CSV (paste or file), copy a text summary of the result, and download the recipe as CSV.
- When no blend can meet the targets, get a plain-language diagnosis of which targets are unachievable before the solver even runs.

> **Exact for the model, not a model of everything.** The solver's answer is the true least-cost blend for the numbers you enter. It is not a sensory model: flavour, bite, colour and processing behaviour enter only through the numeric constraints you set. Preset compositions and prices are illustrative teaching values, not laboratory data or supplier quotes. Chapter 9 states the scope in full.

2. How linear programming works here

The optimiser translates your formulation problem into a linear programme (LP) using three ingredients of its own:

2.1 Variables

There is one variable per ingredient: x_i, the percentage of the blend that ingredient i contributes. With six ingredients there are six variables, and any candidate recipe is a point $(x_1, \ldots, x_6)$ in that space.

2.2 The objective

The quantity to minimise is the cost of one kilogram of blend. If ingredient i costs c_i per kg, the blend cost is the weighted average $\Sigma\, c_i x_i \,/\, 100$ — a linear function of the variables. The app reports it per kg and scaled to your batch size.

2.3 The constraints

Every rule you set becomes one linear constraint:

Rule	Linear form
Blend totals 100%	$\Sigma\, x_i = 100$ (an equality, always present)
Nutrient minimum, e.g. protein ≥ 14%	$\Sigma\, p_i x_i \geq 1400$ (per 100 units of blend)
Nutrient maximum, e.g. fat ≤ 22%	$\Sigma\, f_i x_i \leq 2200$
Ingredient bounds, e.g. ice water ≤ 25%	Folded in exactly by substitution — see Chapter 3
Fat-to-solids ratio window	$\Sigma\, (f_i - r \cdot s_i)\, x_i \leq 0$ or ≥ 0 — algebra in Chapter 5
Group A protein share ≥ q%	$\Sigma_A\, p_i x_i - (q/100) \cdot \Sigma\, p_i x_i \geq 0$ — algebra in Chapter 5

The set of recipes satisfying every constraint is the feasible region. A classical result of LP theory says that if a least-cost feasible recipe exists, one sits at a vertex (corner) of that region — a point where enough constraints meet exactly. That is why optimal recipes so often have several targets met exactly and some ingredients at exactly 0% or at their bounds: the optimum is a corner, and the app's "binding" notes (Chapter 6) tell you which walls of the corner you are pressed against. Those binding constraints are the economically interesting ones — relaxing a binding target is the only kind of change that can reduce cost.

> **Why a real solver matters.** Heuristics and manual tweaking can find a feasible blend; only an exact method can certify that no cheaper feasible blend exists. The simplex method walks from vertex to vertex, improving the cost at every step, and stops only when no improving move remains — at which point the current vertex is provably optimal for the LP as stated.

3. The solver — two-phase simplex, bounds by substitution, tolerances

3.1 Two-phase simplex with Bland's rule

The core implements the simplex method in its two-phase form. Phase 1 introduces artificial variables and minimises their sum to find a first feasible vertex (or to prove none exists — the infeasible verdict). Phase 2 then optimises the true cost from that vertex. Pivot choices follow **Bland's rule** (always the lowest-indexed eligible variable), which is slower than some rules but mathematically guarantees the solver cannot cycle forever on degenerate problems — blends with duplicate or near-duplicate ingredients are exactly such problems, and one of the test cases (Chapter 10) is built on them. A safety cap of 20,000 iterations returns an error status rather than looping indefinitely. The solver can also report unbounded; in this app's formulation that cannot arise from the recipe problem itself (the total is fixed at 100%), but the raw solver detects and reports it, and the test suite exercises that path directly.

3.2 Bounds by substitution

Ingredient min/max bounds are not added as extra constraints. Instead each variable is rewritten as $x_i = lb_i + y_i$ with $0 \le y_i \le ub_i - lb_i$, and every constraint's right-hand side is adjusted by subtracting the known contribution Σ coeff·lb. The substitution is exact — no approximation — and it keeps the constraint matrix small: only the equality, the nutrient rules, the ratio rule and the share rule become rows. After solving, x is recovered by adding the lower bounds back. Minimum-bound behaviour is verified exactly in the test suite: when cost pressure fights a minimum, the ingredient sits at precisely its minimum.

3.3 Tolerances and verification of results

Pivoting uses a tolerance of about 1e-9. After solving, the app does **not** trust the solver's internal bookkeeping for display: the achieved composition (each nutrient, the ratio, the Group A share) is recomputed from the returned recipe vector, and each compliance ✓/✗ and binding flag is derived from that recomputation (binding means within 1e-4 of the target for nutrients and ratio, 1e-3 for the share). Across the test suite, solutions satisfy $|\Sigma x - 100| \le 1.42e\text{-}14$.

4. Entering ingredients — the table and CSV import

4.1 The ingredient table

The table holds 2–12 ingredients (rows can be added and removed). Each row has:

Field	Meaning	Notes
Name	Ingredient label	Used in the recipe, notes and CSV
Group	None / A / B	Group A feeds the protein-share rule (Chapter 5). Group B is a label only in v1.0
Cost per kg	Price in your currency	Currency-agnostic — the app never assumes a currency; use consistent units
Protein / Fat / Moisture / Carbs %	Proximate composition, as-is basis	Must satisfy protein + fat + moisture + carbs ≤ 100% (+ 0.5 tolerance for rounding)
Solids %	Computed: 100 − moisture	Used by the fat-to-solids ratio; not entered directly
Min % / Max %	Inclusion bounds in the blend	Blank min = 0, blank max = 100. Min must be ≤ max

Validation is inline and specific: non-numeric entries, negative values, composition sums above 100%, and min > max are reported against the named row, and optimisation does not run until the table is clean.

4.2 CSV import

For longer ingredient lists, use **Import CSV**: paste text or upload a `.csv` / `.txt` file (read in-page with FileReader — nothing is uploaded anywhere). The column order is:

```
name, cost, protein, fat, moisture, carbs, min, max, group
```

- Separators may be commas, semicolons or tabs; a header row is detected and skipped; blank lines are ignored.
- Only the first five columns are required; carbs, min, max and group default to 0, 0, 100 and None when omitted.
- Unreadable lines are **reported by line number**, never silently dropped — fix them in your file and re-import.
- Choose Replace to swap the whole table or Append to add to it (up to the 12-row limit).

```
name,cost,protein,fat,moisture,carbs,min,max,group
Beef 80/20,8.50,20,20,60,0,0,100,A
Ice water,0.05,0,0,100,0,0,25,None
```

5. Setting targets — including the ratio and protein-share rules

The targets panel takes, for each of protein, fat, moisture and carbohydrate, an optional minimum and/or maximum (leave both blank for "no rule"). It also takes the batch size in kg, used to scale the recipe and total cost. Two further rules deserve explanation, because both look non-linear and both are, on inspection, exactly linear.

5.1 The fat-to-solids ratio

Formulators often specify fat relative to solids rather than to the whole blend (solids $s_i = 100 - \text{moisture}_i$ for each ingredient). The blend ratio is a quotient:

$$\text{fat / solids} = \Sigma\, f_i x_i \;\; / \;\; \Sigma\, s_i x_i$$

Requiring the ratio to be at most r means $\Sigma\, f_i x_i \le r \cdot \Sigma\, s_i x_i$, which rearranges to a single linear constraint:

$$\Sigma\, (f_i - r \cdot s_i)\, x_i \le 0$$

A minimum ratio uses the same coefficients with ≥. The app builds exactly this row (you can see `ratioCoeffs` in the appendix core). One caution follows from the algebra: the ratio

constrains the blend only through its fat and solids totals, so two blends with identical totals but different ingredients are equivalent as far as this rule is concerned — use bounds to steer between them.

5.2 The Group A protein-share rule

Specifications for meat products commonly require that a minimum share of the protein comes from meat (here, Group A) rather than from cheaper protein extenders. If Group A must supply at least q% of total protein:

$$\Sigma_A \, p_i x_i \;\geq\; (q/100) \cdot \Sigma_{all} \, p_i x_i$$

Moving everything to one side gives the linear row the solver actually sees:

$$\Sigma_A \, p_i x_i - (q/100) \cdot \Sigma_{all} \, p_i x_i \geq 0$$

In words: each Group A ingredient's protein counts at $(1 - q/100)$ and every other ingredient's protein counts at $-q/100$. Note the rule is about the share of protein, not the share of the blend — a small amount of a high-protein extender can breach it, which is precisely what the rule exists to catch. In v1.0 the share rule references Group A only; Group B exists as an organising label (a second share rule is a natural v1.1).

6. Reading the results — recipe, cost breakdown, compliance, binding notes

6.1 Status banner

OPTIMAL — the recipe shown is the least-cost blend meeting every rule, with the cost per kg and the batch total. **INFEASIBLE** — no blend of the listed ingredients within their bounds can meet all targets; Chapter 7 explains the diagnosis panel. **ERROR / UNBOUNDED** — reported honestly if the solver's safety limits are ever reached (not expected in normal formulation use).

6.2 The recipe table

One row per ingredient: % of blend, kg in your batch, cost contribution per batch, and % of total cost — so you can see not just what is in the blend but what is driving its price. Ingredients at 0% are shown as 0%: the solver genuinely excluded them (in the burger preset, pork backfat — despite being cheap per kg, its fat load makes it expensive per unit of what the targets allow, a classic LP insight).

6.3 The compliance panel

Each target is listed with the achieved value — recomputed from the recipe, never copied from solver internals (Chapter 3.3) — and a ✓ or ✗. For an optimal solution every row shows ✓; if you ever see a ✗ on an optimal banner, treat it as a bug report worth sending.

6.4 Binding notes

The notes name every target met exactly and every ingredient sitting at its min or max. These are the economically live constraints:

- A **binding nutrient target** is costing you money — relaxing it (if the specification allows) is the only change that can cut cost.
- A **non-binding target** can be tightened somewhat for free, until it becomes binding.
- An ingredient **at its maximum** is one the solver wants more of (typically a cheap diluent like ice water); an ingredient at its **minimum** is one it is forced to carry.

Used this way, the optimiser is not just a recipe generator but a negotiation tool: it prices each clause of a specification.

7. When there is no answer — the infeasibility guide

An infeasible verdict is information, not failure: it says the specification, as written, cannot be met by these ingredients at any proportions. The app helps you find the clash in two stages.

7.1 Pre-checks (run before the solver)

Three exact checks run first, and each names the culprit in plain language:

- **Minimums overflow:** ingredient minimums total more than 100% — the blend cannot even start.
- **Maximums underflow:** ingredient maximums total less than 100% — the blend cannot reach a full batch.
- **Target outside the achievable range:** for each nutrient, the app computes the exact minimum and maximum achievable within the bounds (by a greedy fill: to maximise a nutrient, fill from the highest-content ingredient up to its bound, then the next, and so on). A protein minimum above the achievable maximum — or a fat maximum below the achievable minimum — is named with both numbers, e.g. "Protein minimum target 40% is above the maximum achievable 23.6% from these ingredients within their bounds."

7.2 Joint clashes

Occasionally every single check passes yet the combination is impossible — for example a high protein minimum together with a low fat maximum, where all the protein-rich ingredients are also fatty. The solver then returns infeasible with empty pre-checks. To find

the conflicting pair, relax one target at a time (or widen a bound) and re-run: the target whose relaxation restores feasibility is one side of the clash. The binding-notes habit from Chapter 6 helps here too — near-miss formulations show you which targets were already straining.

7.3 Common causes in practice

Symptom	Likely cause	Fix
Protein target named	Not enough high-protein ingredient allowed (bounds too tight)	Raise the fortifier's max, or lower the protein target
Fat target named	Lean ingredients cannot dilute fat below the ceiling	Add or uncap a lean/water ingredient; raise the fat ceiling
Maximums underflow	Several ingredients capped for sensory reasons	Raise at least one cap so caps total ≥ 100%
Share rule infeasible	Group A protein capacity too small at the required share	Tag more ingredients as A, raise their bounds, or lower q

8. The presets — with the beef burger worked example

Three presets ship with the app. Their compositions and prices are illustrative teaching values — plausible, round-numbered, and labelled as such in the app; they are not laboratory analyses or market quotes.

Preset	Ingredients	Rules	Optimum
Beef burger blend	Beef 80/20, beef trim 50CL, pork backfat, rusk, ice water, soy protein concentrate	Protein ≥ 14%, fat ≤ 22%, moisture ≤ 68%, Group A (meats) ≥ 80% of protein	4.82 per kg — worked in full below
Protein-enriched bread flour	Wheat flour, vital wheat gluten, defatted soy flour (≤ 15%)	Protein ≥ 16%, moisture ≤ 13%	0.66 per kg; soy flour 12.20%, gluten 0%
Teaching example — protein drink mix	Water, sugar, skim milk powder	Protein ≥ 10%, moisture ≤ 20% (both bind)	1.68 per kg; water 19.2%, sugar 53.1%, milk powder 27.8%

8.1 The burger, worked in full

The burger preset (100 kg batch) arrives pre-loaded and pre-optimised. Ingredients and rules:

Ingredient	Cost/kg	Protein	Fat	Moisture	Carbs	Max	Group
Beef 80/20	8.50	20	20	60	0	100	A
Beef trim 50CL	5.00	13	50	36	0	100	A
Pork backfat	2.50	3	88	9	0	60	A
Rusk / breadcrumb	1.20	10	2	10	70	12	—
Ice water	0.05	0	0	100	0	25	—
Soy protein concentrate	3.80	65	1	6	20	8	—

Targets: protein ≥ 14%, fat ≤ 22%, moisture ≤ 68%, Group A protein share ≥ 80%. The optimum (independently re-verified against the shipped solver for this documentation):

Ingredient	% of blend	Note
Beef 80/20	37.28	
Beef trim 50CL	28.80	
Pork backfat	0.00	Excluded — its fat breaks the ceiling
Rusk / breadcrumb	5.45	
Ice water	25.00	At its maximum — the cheapest diluent
Soy protein concentrate	3.47	Limited by the 80% meat-protein share rule

Cost: 4.82 per kg — 481.85 for the 100 kg batch. Achieved composition: protein **14.00%**, fat **22.00%**, moisture 58.49%, fat-to-solids ratio 0.530, Group A protein share **80.00%**. Three constraints bind simultaneously — protein minimum, fat maximum and the share rule — plus ice water at its cap: the recipe is a true LP vertex, pressed against four walls at once. Read it as a story: water is nearly free, so the solver takes all it may; fat is the scarce allowance, spent entirely on the two beefs, leaving none for backfat; soy concentrate is the cheapest protein, but the share rule rations it to 3.47%, forcing the balance of protein to come from beef.

> **Try it:** raise the fat ceiling from 22% to 23% and re-optimise — backfat enters and cost falls. Then restore 22% and lower the share rule from 80% to 70% — soy rises and cost falls again. Each binding constraint has a price, and the app lets you measure it.

9. Honest scope and limitations

- **Linearity is an assumption about the world, not just the maths.** Cost and proximate composition genuinely blend linearly, so within the model the optimum is exact. Anything that does not blend linearly — flavour release, texture, colour,

emulsion stability, cook loss, microbial behaviour — is outside the model and enters only through constraints you impose.

- **No sensory or process model.** A blend can be optimal and inedible. Bounds, ratios and share rules are the tools provided for encoding what the numbers alone cannot say; formulation judgement remains the user's.
- **Preset data is illustrative.** Compositions and prices in the three presets are teaching values. Commercial formulation requires verified laboratory composition for your actual raw materials (which vary by supplier, season and lot) and current supplier prices.
- **Group B is a label only in v1.0.** The protein-share rule references Group A. Group B exists for organising and CSV completeness.
- **Problem size and types.** At most 12 ingredients and the constraint types described. No integer/binary variables (no "either/or" ingredient rules), no multi-period, inventory or supplier minimum-order modelling beyond % bounds; costs are single point values with no quantity price breaks.
- **Pathological data.** Near-duplicate ingredients with wildly different costs make the LP ill-conditioned; Bland's rule plus the 20,000-iteration safety cap guarantee a terminating answer (possibly an honest "error") rather than an infinite loop.
- **Privacy.** No backend is needed or faked: no server, no storage, no API keys — your formulations never leave the browser.

10. How correctness was verified — testing summary

The solver was tested the way a solver must be: the expected answers were derived **by hand, by algebra, before the solver was run** — never read off its own output. The app's PURE CORE (solver, constraint builder, achieved-composition computation, pre-checks, CSV parser, presets) was extracted and exercised in Node through 109 automated checks; the full inline script also passed a syntax check. **Result: 109 / 109 checks passed.** Across all optimal solutions, $|\Sigma x - 100| \leq 1.42e\text{-}14$, and achieved composition and cost were recomputed independently from each solution vector and matched to 1e-9.

Hand-derived case	Expected (by algebra)	Solver
2-ingredient boundary optimum (protein ≥ 20)	(50, 50), 3.50/kg	✓ matched
3-ingredient vertex (protein ≥ 12, fat ≤ 10)	(70, 30, 0), 1.90/kg, protein exactly 12	✓ matched
Maximum bound active	(40, 60), 2.80/kg, bound named	✓ matched
Fat:solids ratio active (≤ 0.25)	(46.667, 53.333), ≈ 2.533/kg, ratio exactly 0.25	✓ matched
Group A share active (≥ 60%)	(71.429, 0, 28.571), ≈ 3.857/kg, share exactly 60%	✓ matched
Minimum bound pinned by substitution	(30, 70), 4.10/kg	✓ matched
Degenerate duplicate ingredients	4.50/kg, no cycling (Bland's rule)	✓ matched
Infeasible nutrient target	Pre-check names Protein	✓ matched
Infeasible bounds (Σmax < 100; Σmin > 100)	Both named by pre-checks	✓ matched
Jointly infeasible targets (pre-checks empty)	Solver-level infeasible verdict	✓ matched
CSV parser (headers, ; / tab separators, defaults, bad lines 3 & 4, empty input, group B)	7 checks	✓ matched
Presets vs hand values (flour 0.661/kg, soy 12.2%; teaching 1.678/kg; burger feasible, share ≥ 80%)	12 checks	✓ matched

The preset figures in Chapter 8 were additionally re-verified for this documentation by running the shipped core directly: burger OPTIMAL at 4.82/kg (481.85 per 100 kg) with protein 14.00%, fat 22.00%, moisture 58.49%, ratio 0.530 and Group A share 80.00%; and the burger with protein minimum raised to 40 returns INFEASIBLE with the pre-check naming Protein above its achievable maximum of 23.6%.

11. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)

Step 1. In the Wix dashboard, open the CMS collection **SaasProducts**.

Step 2. Edit (or add) the item with slug `lp-recipe-optimizer`.

Step 3. Paste the full contents of `index.html` into the **appHtml** field and set the listing fields:

Field	Value
Title	Linear Programming Recipe & Cost Optimizer
Description	Find the least-cost food formulation that meets your protein, fat and moisture targets — a real LP simplex solver with bounds, fat:solids and meat-protein share rules, cost breakdown and compliance checks.
Category	Food Science
Slug	lp-recipe-optimizer
Version	1.0
Published	On

Step 4. Save, open the live app page, and run the smoke test: the page loads with the **Beef burger blend** preset already optimised — expect status **OPTIMAL, 4.82 per kg** (total **481.85** for the 100 kg batch), recipe Beef 80/20 37.28%, Beef trim 50CL 28.80%, Pork backfat 0%, Rusk 5.45%, Ice water 25.00%, Soy protein concentrate 3.47%, and achieved protein 14.00%, fat 22.00%, moisture 58.49%, Group A share 80.00%. Then load the **bread flour** preset → expect **0.66 per kg** with soy flour **12.20%** and gluten 0%. Finally set the burger's protein minimum to 40 → expect **INFEASIBLE** with a pre-check naming Protein above its achievable maximum.

12. Product information and credits

Item	Detail
Product	Linear Programming Recipe & Cost Optimizer, version 1.0
Creator	Dr. Asif Ahmad
Published by	Asif SaaS Studio
Category	Food Science
Format	Single-file web application (index.html, 49,698 bytes); all CSS and JavaScript inline; no build step, no server, no storage, no external requests
Licence	MIT
Method	Two-phase simplex linear programming (Bland's rule), bounds by exact substitution, compliance recomputed from the solution vector

Appendix — the tested core, verbatim

The complete PURE CORE of the shipped app — solver, constraint builder, pre-checks, CSV parser and presets — reproduced exactly as tested (109 / 109 checks). This is the code the documentation above describes.

```
/* ================= PURE CORE — no DOM below this line ================= */
function pcNum(v){ var n = parseFloat(v); return isFinite(n) ? n : 0; }
function pcBlank(v){ return v === null || v === undefined || (typeof v === 'string' && v.trim() === ''); }
function pcFmt(x, d){
  if (d === undefined) d = 2;
  if (!isFinite(x)) return '—';
  var s = (+x).toFixed(d);
  if (s.indexOf('.') >= 0) s = s.replace(/0+$/, '').replace(/\.$/, '');
  if (s === '-0') s = '0';
  return s;
}

/* Two-phase simplex with Bland's rule.
   minimize c·x  subject to  A[i]·x (senses[i]) b[i],  x >= 0.
   senses entries: '<=', '>=' or '='.  Returns {status, x, objective}
   with status 'optimal' | 'infeasible' | 'unbounded' | 'error'. */
function simplexMin(c, A, senses, b){
  var EPS = 1e-9, FTOL = 1e-7;
  var n = c.length, m = A.length, i, j;
  if (m === 0){
    for (j = 0; j < n; j++) if (c[j] < -EPS) return { status: 'unbounded' };
    var x0 = []; for (j = 0; j < n; j++) x0.push(0);
    return { status: 'optimal', x: x0, objective: 0 };
  }
  // Normalise rows so every RHS >= 0 (flipping the sense when negating).
  var rows = [], sns = [], rhs = [];
  for (i = 0; i < m; i++){
    var r = [], s = senses[i], v = +b[i] || 0;
    for (j = 0; j < n; j++) r.push(+A[i][j] || 0);
    if (v < 0){
      v = -v;
      s = (s === '<=') ? '>=' : (s === '>=') ? '<=' : '=';
      for (j = 0; j < n; j++) r[j] = -r[j];
    }
    rows.push(r); sns.push(s); rhs.push(v);
  }
  // Column layout: [x vars | slacks/surpluses | artificials]
  var slackCol = [], artCol = [], basis = new Array(m), isArt = [];
  var N = n;
  for (i = 0; i < m; i++){
    slackCol.push(-1); artCol.push(-1);
    if (sns[i] === '<='){ slackCol[i] = N++; }
    else if (sns[i] === '>='){ slackCol[i] = N++; artCol[i] = N++; }
    else { artCol[i] = N++; }
  }
  for (j = 0; j < N; j++) isArt.push(false);
  var T = [];
  for (i = 0; i < m; i++){
    var row = new Array(N + 1).fill(0);
    for (j = 0; j < n; j++) row[j] = rows[i][j];
    if (slackCol[i] >= 0) row[slackCol[i]] = (sns[i] === '<=') ? 1 : -1;
    if (artCol[i] >= 0){ row[artCol[i]] = 1; isArt[artCol[i]] = true; basis[i] = artCol[i]; }
    else basis[i] = slackCol[i];
    row[N] = rhs[i];
    T.push(row);
  }
  function pivot(pr, pc){
    var prow = T[pr], piv = prow[pc], jj, ii;
    for (jj = 0; jj < prow.length; jj++) prow[jj] /= piv;
    for (ii = 0; ii < T.length; ii++){
      if (ii === pr) continue;
      var f = T[ii][pc];
      if (f !== 0){
        var rr = T[ii];
        for (jj = 0; jj < rr.length; jj++) rr[jj] -= f * prow[jj];
      }
```

```
    }
    basis[pr] = pc;
  }
  function runPhase(cost){
    var iter = 0, ii, jj;
    while (true){
      if (++iter > 20000) return 'error';
      var inBasis = {};
      for (ii = 0; ii < basis.length; ii++) inBasis[basis[ii]] = true;
      var enter = -1; // Bland: smallest index with negative reduced cost
      for (jj = 0; jj < N; jj++){
        if (inBasis[jj]) continue;
        var rc = cost[jj];
        for (ii = 0; ii < T.length; ii++) rc -= cost[basis[ii]] * T[ii][jj];
        if (rc < -1e-9){ enter = jj; break; }
      }
      if (enter === -1) return 'optimal';
      var leave = -1, bestRatio = Infinity, bestBasis = Infinity;
      for (ii = 0; ii < T.length; ii++){
        var a = T[ii][enter];
        if (a > EPS){
          var rv = T[ii][N];
          if (rv < 0 && rv > -1e-7) rv = 0;
          var ratio = rv / a;
          if (ratio < bestRatio - 1e-9 || (Math.abs(ratio - bestRatio) <= 1e-9 && basis[ii] < bestBasis)){
            bestRatio = ratio; bestBasis = basis[ii]; leave = ii;
          }
        }
      }
      if (leave === -1) return 'unbounded';
      pivot(leave, enter);
    }
  }
  var hasArt = false;
  for (i = 0; i < m; i++) if (artCol[i] >= 0) hasArt = true;
  if (hasArt){
    var cost1 = [];
    for (j = 0; j < N; j++) cost1.push(isArt[j] ? 1 : 0);
    var st1 = runPhase(cost1);
    if (st1 !== 'optimal') return { status: st1 };
    var obj1 = 0;
    for (i = 0; i < T.length; i++) obj1 += cost1[basis[i]] * T[i][N];
    if (obj1 > FTOL) return { status: 'infeasible' };
    // Drive any remaining artificial basics out on a real column; rows that
    // are zero in every real column are redundant and dropped.
    var keepRows = [];
    for (i = 0; i < T.length; i++){
      if (isArt[basis[i]]){
        var inB = {};
        for (var q = 0; q < basis.length; q++) inB[basis[q]] = true;
        var pc = -1;
        for (j = 0; j < N; j++){
          if (isArt[j] || inB[j]) continue;
          if (Math.abs(T[i][j]) > 1e-7){ pc = j; break; }
        }
        if (pc >= 0){ pivot(i, pc); keepRows.push(i); }
      } else keepRows.push(i);
    }
    // Rebuild the tableau without artificial columns.
    var colMap = {}, newN = 0;
    for (j = 0; j < N; j++) if (!isArt[j]) colMap[j] = newN++;
    var T2 = [], basis2 = [];
    for (var kr = 0; kr < keepRows.length; kr++){
      var ri = keepRows[kr];
      var nr = new Array(newN + 1).fill(0);
      for (var oj = 0; oj < N; oj++) if (!isArt[oj]) nr[colMap[oj]] = T[ri][oj];
      nr[newN] = T[ri][N];
      T2.push(nr); basis2.push(colMap[basis[ri]]);
    }
    T = T2; basis = basis2; N = newN;
  }
  var cost2 = new Array(N).fill(0);
  for (j = 0; j < n; j++) cost2[j] = c[j];
  var st2 = runPhase(cost2);
  if (st2 !== 'optimal') return { status: st2 };
  var x = new Array(n).fill(0);
  for (i = 0; i < T.length; i++){
```

```
    if (basis[i] < n){
      var val = T[i][N];
      x[basis[i]] = (Math.abs(val) < 1e-7) ? 0 : val;
    }
  }
  var objective = 0;
  for (j = 0; j < n; j++) objective += c[j] * x[j];
  return { status: 'optimal', x: x, objective: objective };
}

/* Exact min/max of sum(coeffs_i * x_i)/100 subject to sum(x)=100 and
   lbs <= x <= ubs — a greedy fill, exact for a single linear form. */
function achievableRange(coeffs, lbs, ubs){
  var sumLb = 0, i;
  for (i = 0; i < coeffs.length; i++) sumLb += lbs[i];
  var rem = 100 - sumLb;
  if (rem < 0) rem = 0;
  function extreme(dir){
    var x = lbs.slice();
    var order = coeffs.map(function(cv, idx){ return idx; });
    order.sort(function(a, b2){ return dir * (coeffs[a] - coeffs[b2]); });
    var r = rem;
    for (var k = 0; k < order.length && r > 1e-12; k++){
      var idx = order[k];
      var add = Math.min(ubs[idx] - lbs[idx], r);
      if (add > 0){ x[idx] += add; r -= add; }
    }
    var s = 0;
    for (var q = 0; q < coeffs.length; q++) s += coeffs[q] * x[q];
    return s / 100;
  }
  return { min: extreme(1), max: extreme(-1) };
}

function computeAchieved(ings, x){
  var P = 0, F = 0, M = 0, C = 0, S = 0, PA = 0, i;
  for (i = 0; i < ings.length; i++){
    var xi = x[i];
    P += ings[i].protein * xi;
    F += ings[i].fat * xi;
    M += ings[i].moisture * xi;
    C += ings[i].carbs * xi;
    S += (100 - ings[i].moisture) * xi;
    if (ings[i].group === 'A') PA += ings[i].protein * xi;
  }
  P /= 100; F /= 100; M /= 100; C /= 100; S /= 100; PA /= 100;
  return {
    protein: P, fat: F, moisture: M, carbs: C, solids: S,
    ratio: (S > 1e-9) ? F / S : (F > 1e-9 ? Infinity : 0),
    groupAProtein: PA,
    groupAShare: (P > 1e-9) ? (PA / P * 100) : null
  };
}

var NUTRIENTS = ['protein', 'fat', 'moisture', 'carbs'];
var NUTRIENT_NAMES = { protein: 'Protein', fat: 'Fat', moisture: 'Moisture', carbs: 'Carbohydrate' };

/* Build the LP from a plain problem object and solve it.
   problem = { ingredients: [{name,cost,protein,fat,moisture,carbs,min,max,group}],
               targets: {proteinMin,proteinMax,fatMin,fatMax,moistureMin,moistureMax,
                         carbsMin,carbsMax,ratioMin,ratioMax,groupAShareMin},
               batchKg }
   Bounds are folded in exactly: x = lb + y, y >= 0, y <= ub - lb. */
function buildAndSolve(prob){
  var ings = prob.ingredients.map(function(g){
    return {
      name: String(g.name || ''), cost: pcNum(g.cost),
      protein: pcNum(g.protein), fat: pcNum(g.fat),
      moisture: pcNum(g.moisture), carbs: pcNum(g.carbs),
      min: pcBlank(g.min) ? 0 : pcNum(g.min),
      max: pcBlank(g.max) ? 100 : pcNum(g.max),
      group: (g.group === 'A' || g.group === 'B') ? g.group : 'None'
    };
  });
  var t = prob.targets || {};
  function tgt(k){ return pcBlank(t[k]) ? null : pcNum(t[k]); }
  var n = ings.length, i;
```

```
  var lbs = ings.map(function(g){ return g.min; });
  var ubs = ings.map(function(g){ return g.max; });
  var sumLb = 0, sumUb = 0;
  for (i = 0; i < n; i++){ sumLb += lbs[i]; sumUb += ubs[i]; }
  // ---- feasibility pre-checks (exact for single forms) ----
  var pre = [];
  if (sumLb > 100 + 1e-7) pre.push('Ingredient minimums total ' + pcFmt(sumLb) + '% — more than the whole 100%
batch. Lower some minimums.');
  if (sumUb < 100 - 1e-7) pre.push('Ingredient maximums total only ' + pcFmt(sumUb) + '% — the blend cannot
reach 100%. Raise some maximums.');
  var ranges = {};
  NUTRIENTS.forEach(function(nut){
    var coeffs = ings.map(function(g){ return g[nut]; });
    var rg = achievableRange(coeffs, lbs, ubs);
    ranges[nut] = rg;
    var mn = tgt(nut + 'Min'), mx = tgt(nut + 'Max');
    if (mn !== null && mn > rg.max + 1e-6) pre.push(NUTRIENT_NAMES[nut] + ' minimum target ' + pcFmt(mn) + '% is
above the maximum achievable ' + pcFmt(rg.max) + '% from these ingredients within their bounds.');
    if (mx !== null && mx < rg.min - 1e-6) pre.push(NUTRIENT_NAMES[nut] + ' maximum target ' + pcFmt(mx) + '% is
below the minimum achievable ' + pcFmt(rg.min) + '% from these ingredients within their bounds.');
  });
  if (pre.length) return { status: 'infeasible', preChecks: pre, ranges: ranges, ingredients: ings };
  // ---- build LP in y-space ----
  var A = [], senses = [], b = [];
  function addRow(coeffs, sense, rhs){ A.push(coeffs); senses.push(sense); b.push(rhs); }
  function baseOf(coeffs){ var s = 0; for (var q = 0; q < n; q++) s += coeffs[q] * lbs[q]; return s; }
  addRow(ings.map(function(){ return 1; }), '=', 100 - sumLb);
  NUTRIENTS.forEach(function(nut){
    var coeffs = ings.map(function(g){ return g[nut]; });
    var base = baseOf(coeffs);
    var mn = tgt(nut + 'Min'), mx = tgt(nut + 'Max');
    if (mn !== null) addRow(coeffs, '>=', 100 * mn - base);
    if (mx !== null) addRow(coeffs, '<=', 100 * mx - base);
  });
  function ratioCoeffs(r){ return ings.map(function(g){ return g.fat - r * (100 - g.moisture); }); }
  var rMax = tgt('ratioMax'), rMin = tgt('ratioMin');
  if (rMax !== null){ var dMax = ratioCoeffs(rMax); addRow(dMax, '<=', -baseOf(dMax)); }
  if (rMin !== null){ var dMin = ratioCoeffs(rMin); addRow(dMin, '>=', -baseOf(dMin)); }
  var share = tgt('groupAShareMin');
  if (share !== null && share > 0){
    var gc = ings.map(function(g){ return (g.group === 'A' ? g.protein : 0) - (share / 100) * g.protein; });
    addRow(gc, '>=', -baseOf(gc));
  }
  for (i = 0; i < n; i++){
    var ub = ubs[i] - lbs[i];
    if (ub < 100 - 1e-12){
      var row = new Array(n).fill(0);
      row[i] = 1;
      addRow(row, '<=', ub);
    }
  }
  var c = ings.map(function(g){ return g.cost; });
  var res = simplexMin(c, A, senses, b);
  if (res.status !== 'optimal'){
    return { status: res.status, preChecks: [], ranges: ranges, ingredients: ings };
  }
  var x = res.x.map(function(y, idx){
    var v = y + lbs[idx];
    return Math.abs(v) < 1e-9 ? 0 : v;
  });
  var costPerKg = 0;
  for (i = 0; i < n; i++) costPerKg += c[i] * x[i];
  costPerKg /= 100;
  var batchKg = pcBlank(prob.batchKg) ? 1 : pcNum(prob.batchKg);
  if (!(batchKg > 0)) batchKg = 1;
  var ach = computeAchieved(ings, x);
  // ---- compliance + binding, recomputed from x ----
  var checks = [];
  NUTRIENTS.forEach(function(nut){
    var mn = tgt(nut + 'Min'), mx = tgt(nut + 'Max');
    if (mn === null && mx === null) return;
    var v = ach[nut];
    checks.push({
      label: NUTRIENT_NAMES[nut], value: v, min: mn, max: mx,
      ok: (mn === null || v >= mn - 1e-6) && (mx === null || v <= mx + 1e-6),
      binding: (mn !== null && Math.abs(v - mn) <= 1e-4) || (mx !== null && Math.abs(v - mx) <= 1e-4)
    });
```

```
  });
  if (rMin !== null || rMax !== null){
    checks.push({
      label: 'Fat : solids ratio', value: ach.ratio, min: rMin, max: rMax, isRatio: true,
      ok: (rMin === null || ach.ratio >= rMin - 1e-6) && (rMax === null || ach.ratio <= rMax + 1e-6),
      binding: (rMin !== null && Math.abs(ach.ratio - rMin) <= 1e-4) || (rMax !== null && Math.abs(ach.ratio -
rMax) <= 1e-4)
    });
  }
  if (share !== null && share > 0){
    var sv = ach.groupAShare;
    checks.push({
      label: 'Group A protein share', value: sv, min: share, max: null, isShare: true,
      ok: sv !== null && sv >= share - 1e-4,
      binding: sv !== null && Math.abs(sv - share) <= 1e-3
    });
  }
  var boundNotes = [];
  for (i = 0; i < n; i++){
    if (lbs[i] > 0 && Math.abs(x[i] - lbs[i]) <= 1e-6) boundNotes.push(ings[i].name + ' sits at its minimum (' +
pcFmt(lbs[i]) + '%).');
    if (ubs[i] < 100 && Math.abs(x[i] - ubs[i]) <= 1e-6) boundNotes.push(ings[i].name + ' sits at its maximum ('
+ pcFmt(ubs[i]) + '%).');
  }
  return {
    status: 'optimal', x: x, costPerKg: costPerKg, totalCost: costPerKg * batchKg,
    batchKg: batchKg, achieved: ach, checks: checks, boundNotes: boundNotes,
    ingredients: ings, ranges: ranges
  };
}

/* Tolerant ingredient CSV parser.
   Columns: name,cost,protein,fat,moisture[,carbs,min,max,group].
   Comma/semicolon/tab separated, optional header row, blank lines skipped.
   Unreadable lines are reported with their 1-based line number. */
function parseIngredientsCSV(text){
  var out = { rows: [], errors: [] };
  if (!text || !String(text).trim()){
    out.errors.push({ line: 0, message: 'No data — the input was empty.' });
    return out;
  }
  var lines = String(text).replace(/\r\n?/g, '\n').split('\n');
  var firstContentSeen = false;
  for (var li = 0; li < lines.length; li++){
    var raw = lines[li];
    if (!raw.trim()) continue;
    var delim = raw.indexOf('\t') >= 0 ? '\t' : (raw.indexOf(';') >= 0 ? ';' : ',');
    var f = raw.split(delim).map(function(s){ return s.trim(); });
    if (!firstContentSeen){
      firstContentSeen = true;
      if (f.length >= 2 && f[1] !== '' && isNaN(parseFloat(f[1]))) continue; // header row
    }
    var lineNo = li + 1;
    if (f.length < 5){
      out.errors.push({ line: lineNo, message: 'Expected at least 5 columns (name,cost,protein,fat,moisture) —
found ' + f.length + '.' });
      continue;
    }
    if (!f[0]){
      out.errors.push({ line: lineNo, message: 'Missing ingredient name.' });
      continue;
    }
    var nums = [], bad = -1, k;
    for (k = 1; k <= 4; k++){
      var v = parseFloat(f[k]);
      if (!isFinite(v)){ bad = k; break; }
      nums.push(v);
    }
    if (bad >= 0){
      out.errors.push({ line: lineNo, message: 'Column ' + (bad + 1) + ' ("' + f[bad] + '") is not a
number.' });
      continue;
    }
    function opt(idx, dflt){
      if (f.length > idx && f[idx] !== '' && f[idx] !== undefined){
        var v2 = parseFloat(f[idx]);
        return isFinite(v2) ? v2 : dflt;
```

```
      }
      return dflt;
    }
    var grp = 'None';
    if (f.length > 8 && f[8]){
      var g = f[8].toUpperCase();
      if (g === 'A' || g === 'B') grp = g;
    }
    out.rows.push({
      name: f[0], cost: nums[0], protein: nums[1], fat: nums[2], moisture: nums[3],
      carbs: opt(5, 0), min: opt(6, 0), max: opt(7, 100), group: grp, line: lineNo
    });
  }
  return out;
}

/* Presets — illustrative teaching data (typical compositions, round prices). */
var PRESETS = {
  burger: {
    label: 'Beef burger blend',
    batchKg: 100,
    ingredients: [
      { name: 'Beef 80/20', cost: 8.50, protein: 20, fat: 20, moisture: 60, carbs: 0, min: 0, max: 100, group:
'A' },
      { name: 'Beef trim 50CL', cost: 5.00, protein: 13, fat: 50, moisture: 36, carbs: 0, min: 0, max: 100,
group: 'A' },
      { name: 'Pork backfat', cost: 2.50, protein: 3, fat: 88, moisture: 9, carbs: 0, min: 0, max: 60, group:
'A' },
      { name: 'Rusk / breadcrumb', cost: 1.20, protein: 10, fat: 2, moisture: 10, carbs: 70, min: 0, max: 12,
group: 'None' },
      { name: 'Ice water', cost: 0.05, protein: 0, fat: 0, moisture: 100, carbs: 0, min: 0, max: 25, group:
'None' },
      { name: 'Soy protein concentrate', cost: 3.80, protein: 65, fat: 1, moisture: 6, carbs: 20, min: 0, max:
8, group: 'None' }
    ],
    targets: { proteinMin: 14, fatMax: 22, moistureMax: 68, groupAShareMin: 80 }
  },
  flour: {
    label: 'Protein-enriched bread flour',
    batchKg: 100,
    ingredients: [
      { name: 'Wheat flour', cost: 0.60, protein: 11, fat: 1.2, moisture: 12, carbs: 72, min: 0, max: 100,
group: 'None' },
      { name: 'Vital wheat gluten', cost: 3.20, protein: 75, fat: 1, moisture: 8, carbs: 10, min: 0, max: 100,
group: 'None' },
      { name: 'Defatted soy flour', cost: 1.10, protein: 52, fat: 1.5, moisture: 8, carbs: 30, min: 0, max: 15,
group: 'None' }
    ],
    targets: { proteinMin: 16, moistureMax: 13 }
  },
  teaching: {
    label: 'Teaching example — protein drink mix',
    batchKg: 10,
    ingredients: [
      { name: 'Water', cost: 0.02, protein: 0, fat: 0, moisture: 100, carbs: 0, min: 0, max: 100, group:
'None' },
      { name: 'Sugar', cost: 0.80, protein: 0, fat: 0, moisture: 0, carbs: 100, min: 0, max: 100, group:
'None' },
      { name: 'Skim milk powder', cost: 4.50, protein: 36, fat: 1, moisture: 3, carbs: 52, min: 0, max: 100,
group: 'None' }
    ],
    targets: { proteinMin: 10, moistureMax: 20 }
  }
};
/*PURE-CORE-END*/
```